Modelska analiza 1: Nelinearna optimizacija

Žan Ambrožič (28262032)

18. oktober 2026

1 Naloga

V nalogi bomo reševali Thomsonov problem, ki v splošnem ni analitično rešljiv. Na prevodno kroglo nanesemo N enakih nabojev in iščemo optimalno razporeditev (najmanjša energija). Za reševanje bomo uporabili različne optimizacijske metode in primerjali rezultate ter njihovo učinkovitost. V drugem delu bomo ponovili še problem optimalne vožnje skozi semafor, ki mu sedaj lahko dodamo različne omejitve, ki jih je bilo v prejšnji različici težje upoštevevati.

2 Thomsonov problem

Optimalna rešitev Thomsonovega problema je taka razporeditev N enakih nabojev na površini krogle, pri kateri je elektrostatska energija minimalna. Za potrebe minimizacije lahko izpustimo konstantne predfaktorje in definiramo elektrostatsko energijo kot:

$$V = \sum_{i<j} \frac{1}{d_{ij}}; \qquad d_{ij} = |\mathbf{r}_i - \mathbf{r}_j| \,. \tag{1}$$

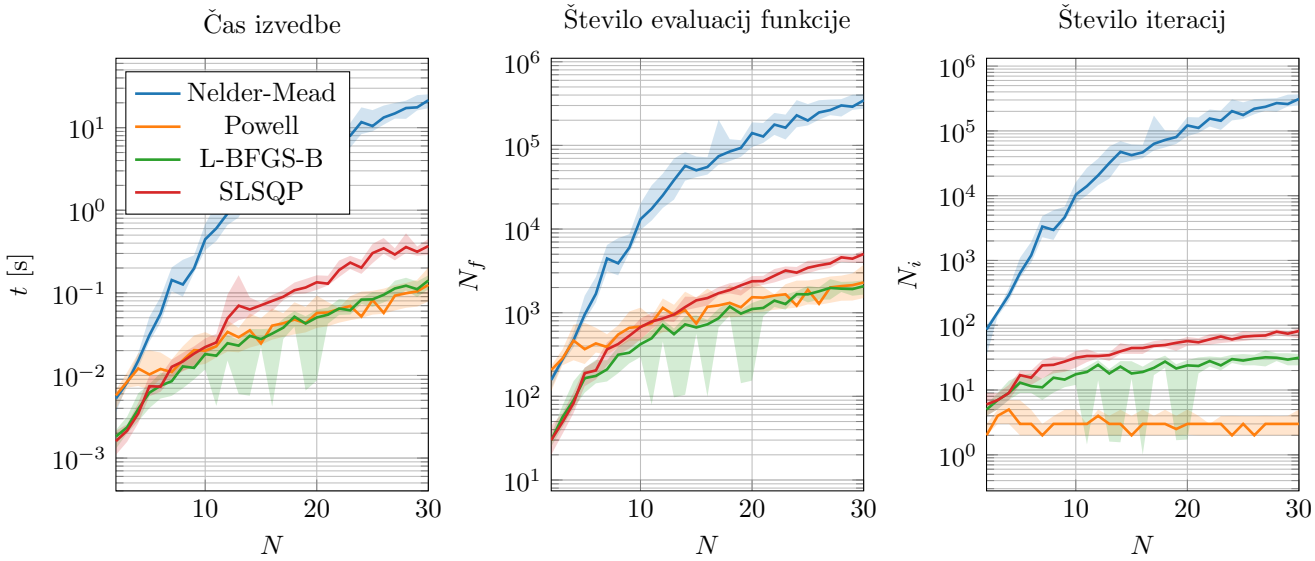
Dobra metrika za oceno kakovosti razporeditve je tudi velikost dipolnega momenta, ki ga lahko izračunamo kot:

$$\mathbf{p} = \sum_{i=1}^{N} \mathbf{r}_i. \tag{2}$$

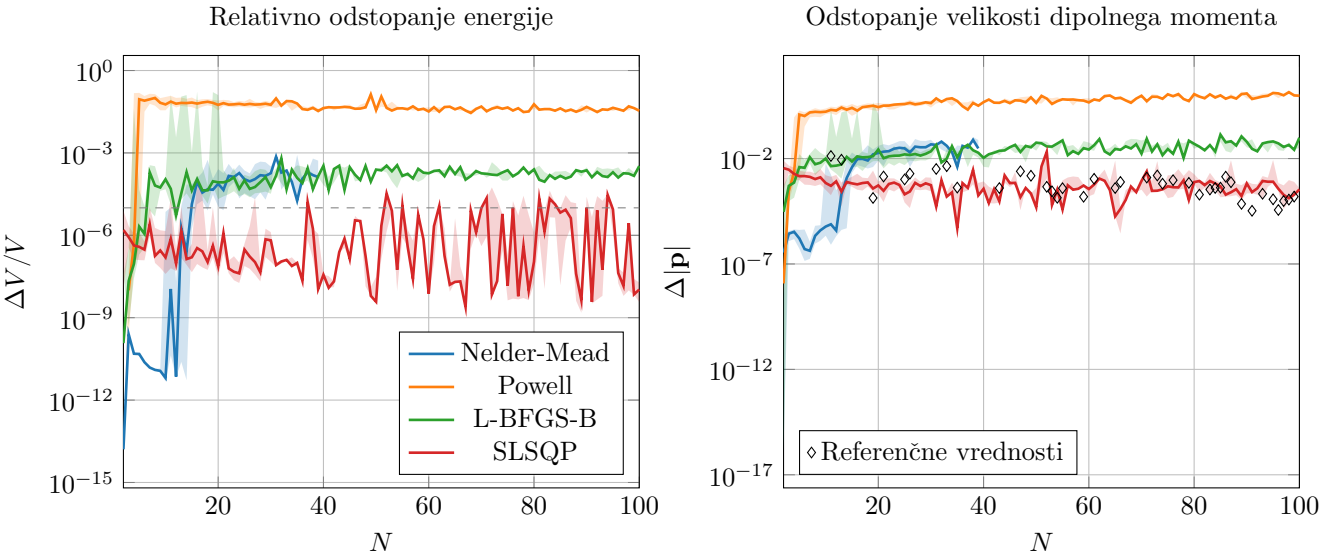
Dipolni moment nam tudi pove, ali je razporeditev simetrična ali ne. Za referenco imamo na razpolago energije in dipolne momente optimalnih razporeditev manjših števil nabojev [1]. Ker začetne položaje nabojev izbiramo naključno, so lahko rezultati testov zelo različni – zato vsako metodo za vsako število nabojev poženemo dvajsetkrat na različnih naključnih začetnih razporeditvah (ki lahko najdejo različne minimume, ki so lahko tudi daleč od globalnega). Na grafih prikažemo mediane in intervale 1σ. Ker se v redkih primerih zgodi, da algoritem pri dovoljenem številu evalvacij funkcije ($2N \cdot 10^5$ oz. 100 iteracij pri SLSQP) ne najde rešitve, te primere izključimo iz statistike. Vse metode so testirane z nastavitvijo tolerance 10^{-5}.

Primerjali bomo nekatere od metod iz `scipy.optimize.minimize`, ki omogočajo nastavitev meja območja minimizacije:

- Nelder-Mead – simpleksna, ne uporablja odvodov, počasna, a robustna;
- Powell – uporablja konjugirane smeri, ne uporablja odvodov;
- L-BFGS-B – uporablja prvi odvod in za razliko od običajne BFGS omogoča nastavitev meje območja minimizacije;
- SLSQP – uporablja metodo najmanjših kvadratov (in s tem tudi prve odvode).



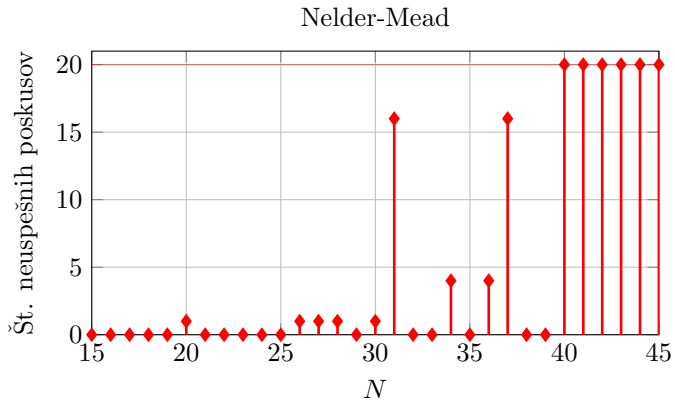
Graf 1: Časovna in računska zahtevnost različnih minimizacijskih metod z nastavitvijo tolerance 10^{-5}.



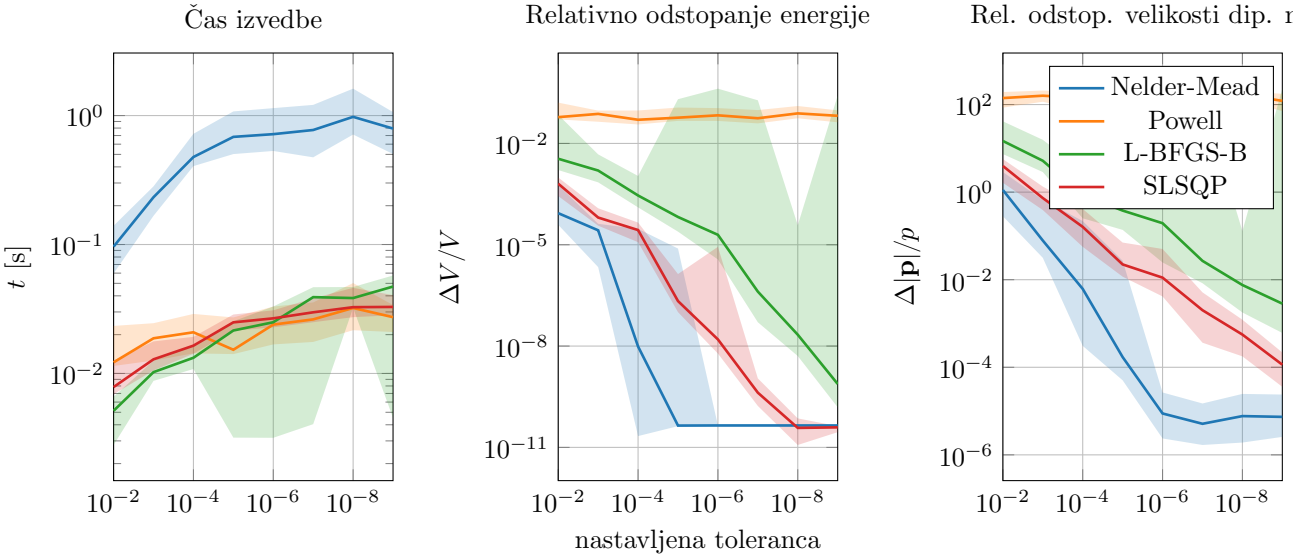
Graf 2: Odstopanje energije pri različnih minimizacijskih metodah z nastavitvijo tolerance 10^{-5}. S prekinjeno črto je označena meja tolerance (10^{-5}), ki za dipolni moment ni prava metrika, saj minimizator računa le energijo, moment pa izračunamo šele naknadno. Opazimo tudi, da so odstopanja velikosti dipolnih momentov le pri manjših N primerljiva z referenčnimi vrednostmi, za večje N pa so odstopanja bistveno večja.

Najbolj časovno in računsko potratna je metoda Nelder-Mead, vendar za manjše N daje najboljše rezultate. Metodi Powell in L-BFGS-B sta hitrejši, vendar so njuni rezultati nezanesljivi (sploh Powell) – vrnejo rešitve, ki niso globalni minimumi. Boljša in ne toliko počasnejša od prejšnjih dveh je metoda SLSQP, ki edina ostaja pod nastavljeno mejo tolerance v energiji (tj. najde globalni minimum ali minimum, ki je skoraj tako nizek, kot globalni) in tudi ohranja odstopanje dipola v velikostnem redu prave vrednosti dipolnega momenta, toda nobena od metod pri nastavljeni natančnosti ne opisuje dipolnega momenta dovolj dobro, da bi bili rezultati zanesljivi.

Velja omeniti, pri večjih N (nad 10) začne SLSQP opazno (tako, da je vidno na prikazovalniku, ki se osveži vsake pol sekunde) uporabljati vsa jedra (kar na 16-jedernem procesorju ni zanemarljiv pospešek v primerjavi z Nelder-Mead, ki teče le na enem jedru), medtem ko L-BFGS-B pri privzetih nastavitvah izkoristi približno dve tretjini jeder (10/16).

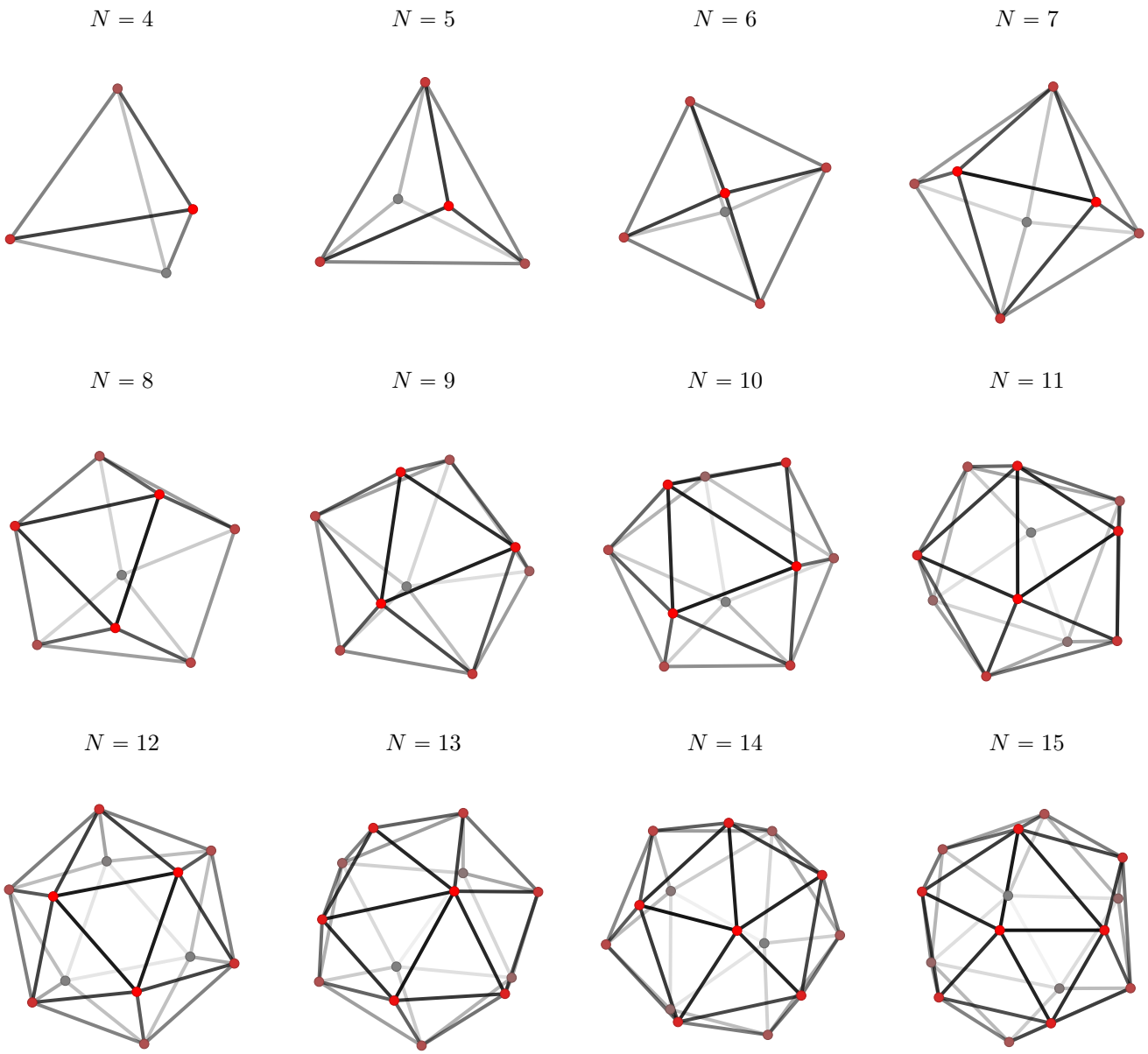


Graf 3: Število neuspešnih poskusov minimizacije (minimizacija ne uspe, če algoritem ne najde rešitve pred ustavljalnim pogojem, kar smo za Nelder-Mead nastavili pri $2N \cdot 10^5$ evalvacijah funkcije). Vseh poskusov je 20. Na grafu je izbrano območje prehoda (za manjše N so bili vsi poskusi uspešni, za večje nobeden), prikazana pa je le metoda Nelder-Mead, saj preostale najdejo rešitve v vseh poskusih (pri danih omejitvah).



Graf 4: Primerjava časovne zahtevnosti in natančnosti minimizacije za $N = 11$ ($V_{11} = 40{,}596\,450\,51$, $|\mathbf{p}| = 0.013219635$). Powell očitno kljub nekoliko daljšemu računskemu času ne izboljša natančnosti, medtem ko ostale metode dajejo boljše rezultate pri nižjih tolerancah (postopek je odvisen od metode in zaporedje operacij pri Nelder-Mead metodi je tako, da že pri nastavitvi tolerance 10^{-5} zadanemo ob mejo končne numerične natančnosti).

Graf odvisnosti odstopanj in časa računskih časov v odvisnosti od tolerance nam povedo več o metodah: najprej opazimo, da pri Nelder-Mead nima smisla nastavljati tolerance pod 10^{-6}, saj se tam tudi natančnosti dipolnega momenta ne izboljša več (podanega imamo na 8 mest, 6 je veljavnih pri rezultatu). Če nas zanima le energija, nam bo Nelder-Mead dal najboljši približek že pri toleranci 10^{-5}. Powell nam sicer da oceno energije, vendar nam odstopanje dipolnega momenta pove, da nismo v globalnem minimumu in rešitev ni popolnoma optimalna – če bi bila, bi se bolje skladala z referenčnimi vrednostmi, vendar so odstopanja pri vseh nastavitvah tolerance okoli faktorja 100. Razen Powellove metode vse izboljšajo točnost rezultatov z nižjimi tolerancami, temu primerno pa se računski čas nekoliko podaljša.



Slika 1: Prikaz nekaj razporeditev nabojev po površini krogle za manjše N za minimizirano energijo (SLSQP, toleranca 10^{-9}). Rdeča oglišča in intenzivne stranice so bliže, medtem ko so siva oglišča in svetlejše stranice bolj oddaljene. Oglišča so zaradi vidnosti narisana v ospredju (mestoma lahko izgleda, da gre stranica za ogliščem)

Na sliki prikazane razporeditve za nekatere N (npr. 8) so nekoliko drugačne, kot bi sprva pričakovali (za $N = 8$ najprej pomislimo na kocko, a se izkaže, da ni najoptimalenjša razporeditev, saj lahko eno osnovno ploskev zarotiramo za 45° in tako povečamo razdalje med naboji – pri tem se seveda malo povečata tudi osnovni ploskvi). Od platonskih teles najdemo vse tri, ki jih sestavljajo trikotniki (tetraeder, oktaeder in ikozaeder), medtem ko tudi dvanajsterca (dodekaedra) ni, saj je iz petkotnikov, kar pa s stališča povprečne razdalje med oglišči ni optimalno.

2.1 Möbiusov trak

Möbiusov trak je zanimiv, saj je na enem delu trak vodoraven, na drugem pa pokončen, geometrija zato ni simetrična. Naboje lahko poskusimo optimalno razporediti po površini, ki jo parametriziramo kot:

$$x = (R + v\cos(u/2))\cos(u), \qquad y = (R + v\cos(u/2))\sin(u), \qquad z = v\sin(u/2), \tag{3}$$

kjer je R polmer kroga, ki ga opisuje sredina traku (vzamemo $R = 1$), v je koordinata vzdolž širine (vzamemo $v \in [-1/4, 1/4]$), u pa je koordinata vzdolž dolžine (vzamemo $u \in [0, 2\pi]$). Začetno razporeditev zopet generiramo naključno čez celotno definicijsko območje, za minimizacijo uporabimo metodo SLSQP s toleranco 10^{-9}.

$N = 8$ $N = 10$ $N = 12$ $N = 14$



$N = 16$ $N = 18$ $N = 20$ $N = 22$



$N = 24$ $N = 26$ $N = 28$ $N = 30$



$N = 32$ $N = 34$ $N = 36$ $N = 38$



Slika 2: Optimalne razporeditve N enakih nabojev na Möbiusovem traku. Sredina traku je v ravnini xy, barva nabojev ponazarja z koordinato. Naboji so zaradi vidnosti narisani v ospredju.

Opazimo, da se naboji najprej razporejajo bolj po zunanji strani traku (tako so najdlje eden od drugega), najkasneje pa na notranjem robu dela, kjer je trak najbolj poravnan z ravnino xy (in v smeri proti pokončnemu delu), kar je smiselno, saj je ta rob od vseh najbližje preostanku traku, kjer so naboji.

3 Semafor

Brezdimenzijske količine smo v prvi nalogi uvedli kot:

$$u = \frac{v}{v_0}, \qquad \tau = \frac{t}{T}, \qquad l = \frac{D}{v_0 T}, \qquad u_f = \frac{v_f}{v_0}. \tag{4}$$

Tokrat pa namesto analitičnega računanja z integrali interval diskretiziramo. Ker τ teče med 0 in 1, ob delitvi na N enakih delov dobimo časovne intervale $\Delta\tau = 1/N$. Funkcijo za minimizacijo nastavimo kot:

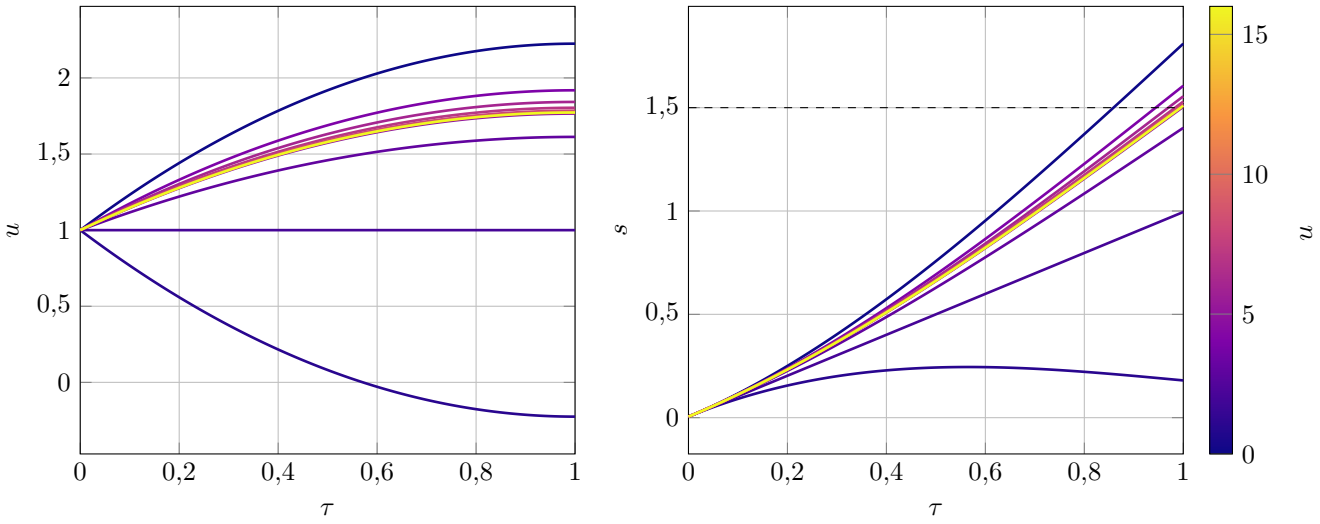
$$L_0 = \sum_{i=1}^{N} \left(\frac{u_i - u_{i-1}}{\Delta\tau} \right)^2 \Delta\tau = \sum_{i=1}^{N} (u_i - u_{i-1})^2 \cdot N. \tag{5}$$

3.1 Iskanje λ

Lahko poskusimo z iskanjem prave vrednosti λ za minimizacijo funkcije

$$L_\lambda = L_0 + \lambda \left(\frac{u_0 + u_N}{2} + \sum_{i=1}^{N-1} u_i \right) N^{-1}, \tag{6}$$

pri čemer za vsako vrednost λ poiščemo optimalno rešitev in nato preverimo, ali je vez izpolnjena. Z bisekcijo lahko najdemo λ, ki izpolni vez in vrnemo optimalen potek hitrosti pri tej vrednosti.



Graf 5: Iskanje optimalne λ z bisekcijo za $l = 1{,}5$ pri nastavljeni toleranci $\lambda \pm 0{,}001$. n predstavlja korak bisekcije.

Kot je razvidno z grafa, najdemo pravo vrednost λ, ki zadosti vezi razdalje do semaforja, vendar ima ta pristop dve omejitvi: bisekcija je počasna in pri vsaki evalvaciji moramo poiskati optimalno rešitev pri dani vrednosti λ, prav tako pa lahko nastavljamo le točno vrednost vezi in ne moremo uporabiti npr. vezi, ki bi dovoljevala npr. vožnjo skozi zeleno nekaj časa po njenem prižigu, preprečevala pa le vožnjo skozi rdečo. Poleg tega ima `scipy.optimize.minimize` za metodo SLSQP, ki jo uporabljamo, vgrajeno možnost nastavitve vezi ali omejitev, kar nam prihrani pisanje dodatne kode za iskanje λ, prav tako je v splošnem verjetno implementirana učinkoviteje (glede na čase izvajanja pri obeh pristopih z isto nastavljeno natančnostjo), zato bomo v nadaljevanju uporabili to možnost ali pa bomo vez implementirali kot mehko v minimizacijsko funkcijo, kar nam omogoča tudi različne obtežitve posameznih omejitev.

3.2 Direktna minimizacija

Vez lahko pri nekaterih metodah (npr. SLSQP) sicer nastavimo, lahko pa jo uvedemo tudi kot mehko (numerična integracija – trapezno):

$$L_1 = \exp\left[\alpha_1 \left(\left[\frac{u_0 + u_N}{2} + \sum_{i=1}^{N-1} u_i \right] N^{-1} - l \right)^2 \right]. \tag{7}$$

Pri tem velja omeniti, da kaznujemo razdaljo od semaforja in ne tega, koliko časa prej ali kasneje prečkamo semafor, kar ni enako, če se hitrost tik pred koncem spreminja (kaj štejemo kot območje pred koncem je deloma odvisno od nastavljenih uteži in robnega pogoja).

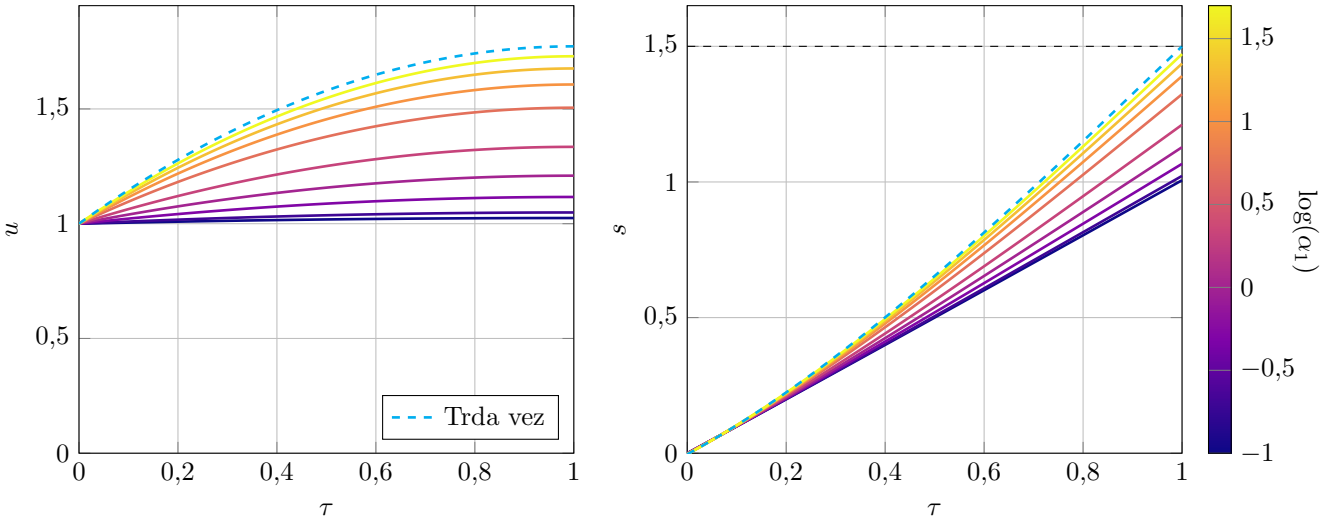
Na podoben način lahko tudi omejimo največjo in najmanjšo dovoljeno hitrost (nekatere metode to že same omogočajo, sicer pa mehko):

$$L_2 = \sum_{i=1}^{N} \left[e^{\alpha_2 (u_i - u_{\max})} + e^{\alpha_3 (u_{\min} - u_i)} \right]. \tag{8}$$

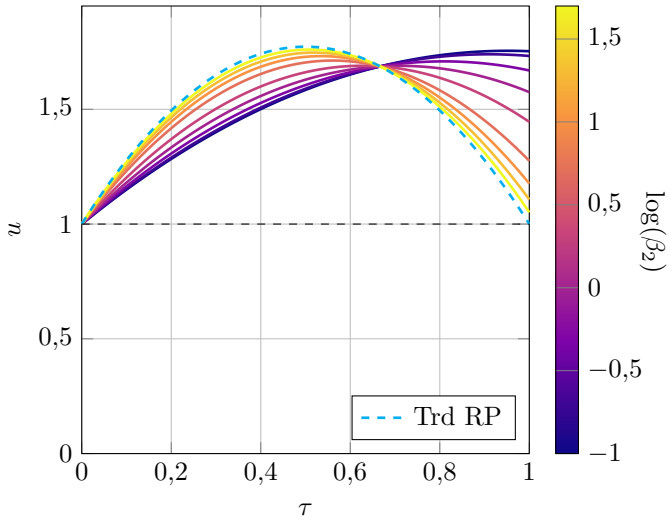
Na ta način lahko nastavljamo tudi robne pogoje (za u_0 in u_N uporabimo mehko vez, ki ima za spodnjo in zgornjo mejo enaki vrednosti, lahko pa uporabimo tudi močnejšu utež β, ki bo bolj kaznovala odstopanje od želene vrednosti):

$$L_{\mathrm{RP}} = e^{\beta_1 (u_0 - 1)^2} + e^{\beta_2 (u_N - u_f)^2}. \tag{9}$$

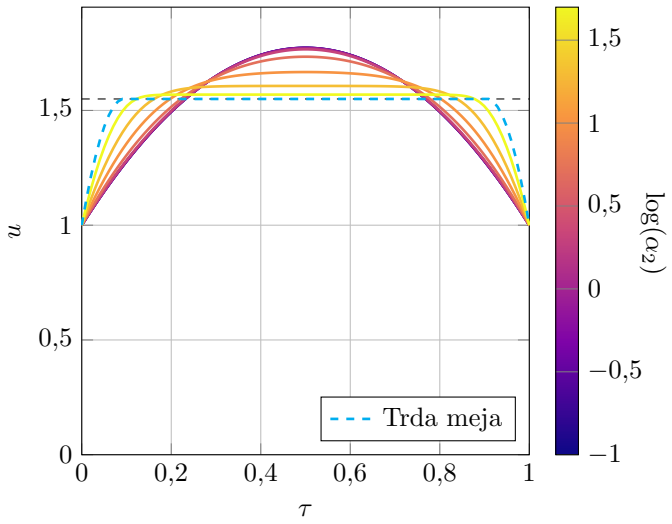
Robne pogoje lahko sicer vsiljujemo tudi tako, da ob vsaki evalvaciji funkcije nastavimo vrednosti na želene. Minimizacija seveda ne bo upoštevala opisanih pogojev popolnoma natančno – kako pomemben je kakšen od njih, nastavimo z izbiro uteži α_x. Pri tem moramo upoštevati, da je povprečen kvadrat pospeška (naša mera za nelagodje) neobtežen (oz. je utež 1), zato prevelike uteži lahko povzročijo manj udobno vožnjo.



Graf 6: Vpliv uteži α_1 (utež vezi) na minimizirano funkcijo $L_0 + L_1$ (začetna hitrost nastavljena kot trd roben pogoj, končna prosta) ob zahtevi vezi $l = 1{,}5$. Pri manjših α_1 dominira udobje (torej sploh ne pospešujemo), medtem ko pri večjih začnemo upoštevati tudi vez, potek hitrosti pa je tak, da pri tem tudi minimiziramo nelagodje. Opazimo, da je prost desni robni pogoj enak naravnemu pogoju iz prve naloge.



Graf 7: Vpliv uteži β_2 (utež končne hitrosti $u_f = 1$) na minimizirano funkcijo $L_0 + L_{\text{RP}}$ (začetna hitrost nastavljena kot trd robni pogoj) ob zahtevi vezi $l = 1{,}5$ (trda vez).



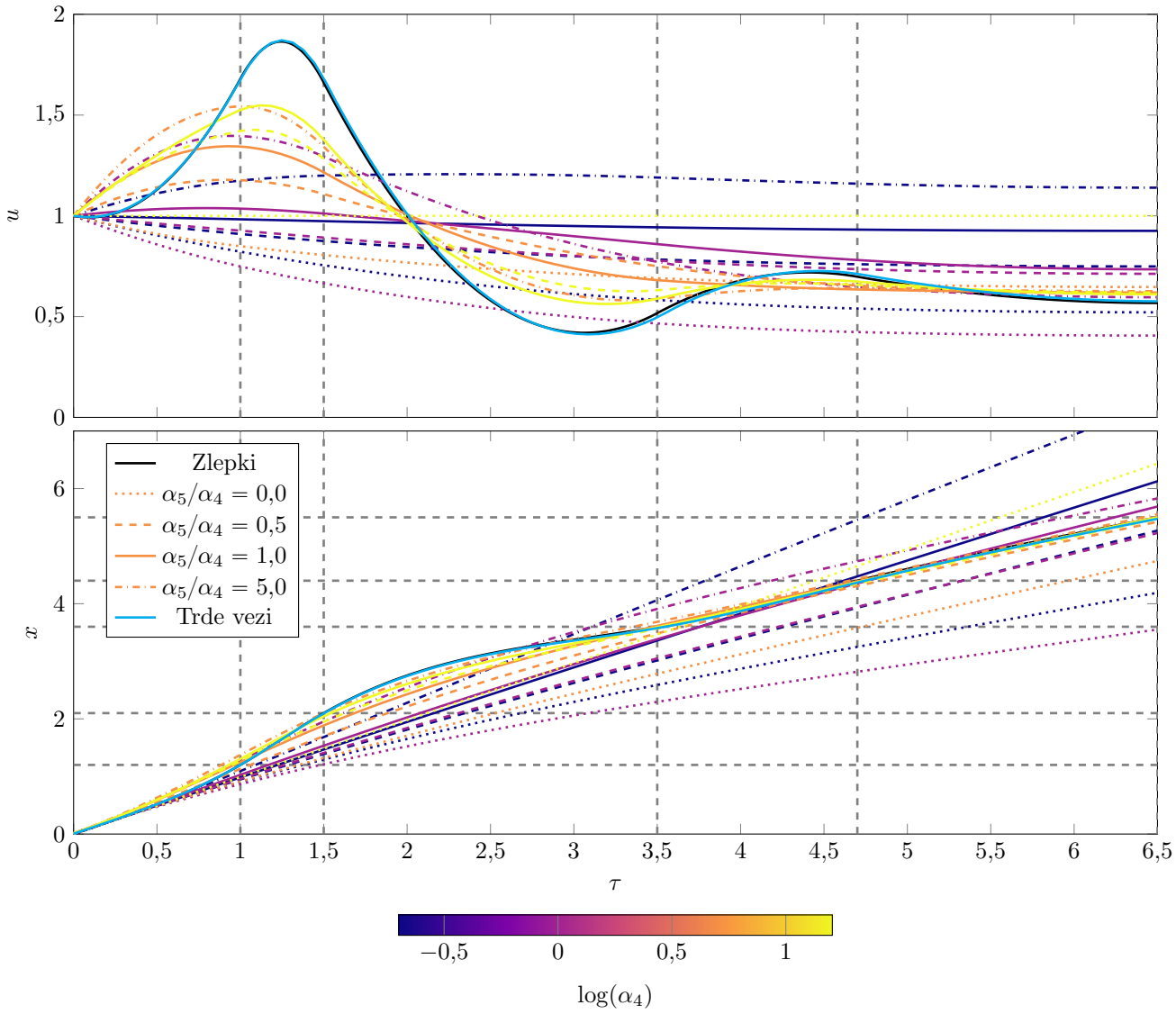
Graf 8: Vpliv uteži α_2 (utež zg. meje) na minimizirano funkcijo $L_0 + L_2$ (začetna in končna hitrost nastavljeni kot trda robna pogoja) ob zahtevi vezi $l = 1{,}5$ (trda vez) in $u_{\max} = 1{,}55$.

3.3 Zaporedni semaforji

Za zaporedne semaforje namesto enega pogoja vezi v tem primeru optimiziramo več mehkih vezi. Pri naši implementaciji je najlažje določiti časovne intervale in gledati ali smo ob prižigu zelene luči pred ali za semaforjem. Kot sem to omenil že v prvi nalogi, je smiselno bolj kaznovati vožnjo skozi rdečo kot pa malo zamude pri prečkanju zelene. Eden od načinov je ta, da med časovno enakomerno razporejenimi v_i določimo indekse i_k, pri katerih bi morali prečkati semaforje med segmenti, ki jih označimo s $k \in \{1, ..., K\}$:

$$L_3 = \sum_{k=1}^{K} \left[\exp\left(\alpha_4 \left[\frac{v_0 + v_{i_k}}{2N} + \sum_{i=0}^{i_k} \frac{v_i}{N} - \sum_{j=1}^{k} l_j \right] \right) + \exp\left(\alpha_5 \left[\sum_{j=1}^{k} l_j - \sum_{i=0}^{i_k} \frac{v_i}{N} - \frac{v_{i_0} + v_{i_k}}{2N} \right] \right) \right], \tag{10}$$

tako za močnejše kaznovanje vožnje skozi rdečo izberemo $\alpha_4 > \alpha_5$.



Graf 9: Primer vožnje skozi zaporedne semaforje (z narisano rešitvijo z zlepki iz prve naloge). Odstopanje med zlepki in rešitvijo s tredo vezjo je posledica omejene natančnosti diskretizacije in se s finejšo delitveijo manjša (na grafu je časovna skala razdeljena na 300 točk). Vezi lahko implementiramo tudi v minimizacijsko funkcijo (10), v kateri α_4 predstavlja eksponentno utež za kaznovanje vožnje skozi rdečo, medtem ko α_5 predstavlja eksponentno utež za kaznovanje zamude pri prečkanju zelene. Pri mehkih utežeh je delitev časovne skale 100 točk.

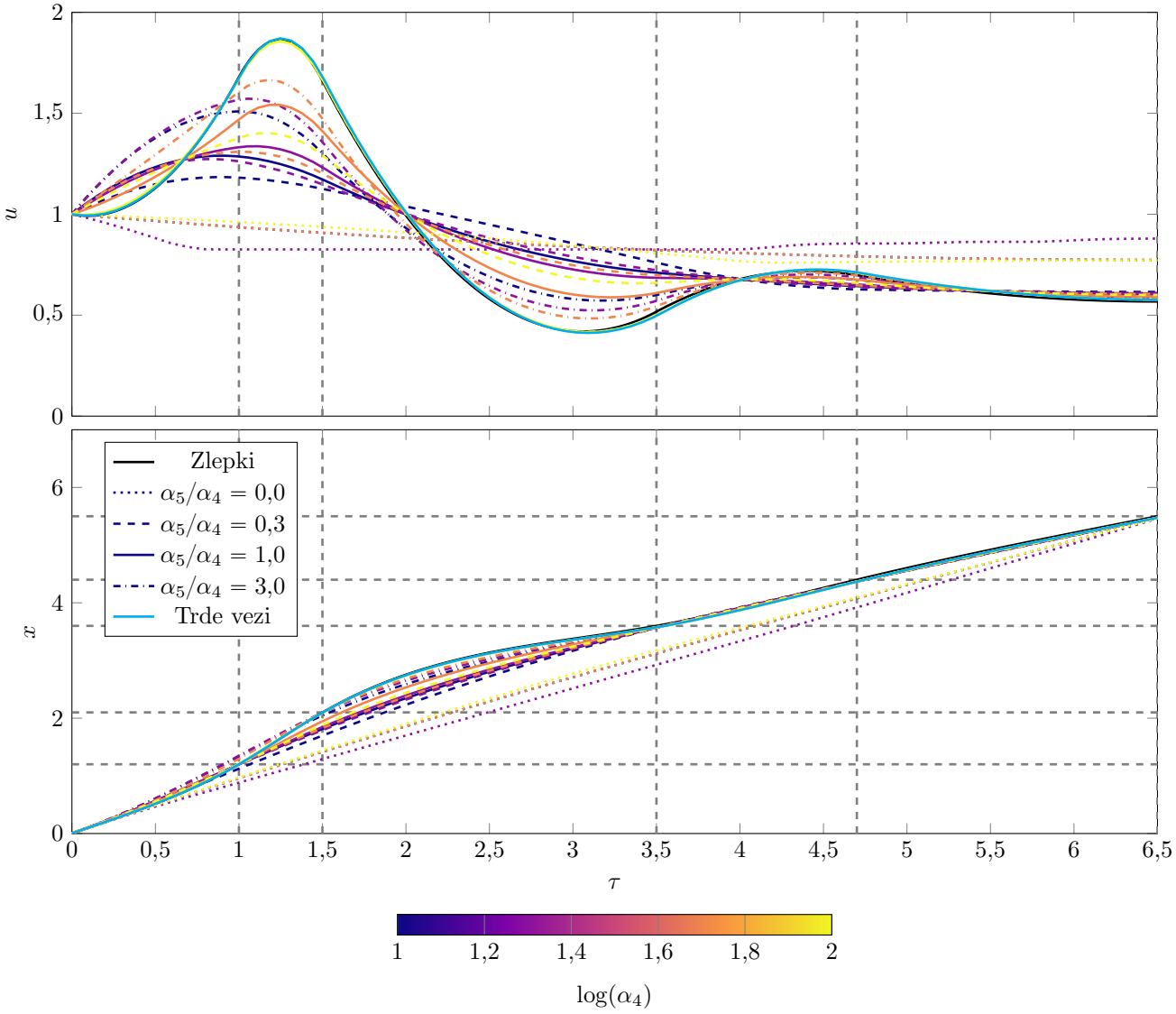
Prvo, kar opazimo pri mehkih vezeh je, da je začetni pospešek precej drugačen kot pri optimalno udobni rešitvi, saj z dodatkom eksponentnih členov vezi efektivno spremenimo sam funkcional in redefiniramo udobje oz. nelagodje. Nato pri različno velikih utežeh in razmerjih med njimi lahko opazimo posledice njihovega vpliva. S črta-pika so narisane hitrosti in poti za funkcional, ki bolj kaznuje zamudo prečkanja pri zeleni, kot pa prečkanje še skozi rdečo – če gledamo prvi semafor ($\tau = 1$ oz. $l = 0{,}7$), ga prečkamo še preden se prižge zelena (oranžna črta, ki ima največji uteži, rumene ni, saj v eksponentu dobimo preveliko številko). Tudi pri blažjih utežeh sicer zamudimo prvi semafor, vendar je pospešek nato pri ostalih veliko manjši, ker je kazen za prezgodnje prečkanje manjša od kazni za pozno.

Po drugi strani bolj realistične uteži, ki kaznujejo prezgodnje prečkanje raje zapeljejo skozi prvi semafor, ko že nekaj časa gori zelena, kot pa da bi peljali skozi rdečo. Rešitve z večanjem uteži sicer postajajo bolj podobne rešitvama z zlepki oz. s trdimi vezmi, vendar jih ne dosežemo (potrebovali bi še večje uteži), ker pri večjih odstopanjih, ki se med optimizacijo pojavijo, prekoračimo največje dovoljeno število, pri manjših odstopanjih pa je kazen kljub temu še vedno majhna.

Ker imamo pri postopku minimizacije težave s prevelikimi eksponentnimi členi, lahko poskusimo še z drugimi funkcijami. Ena od možnosti, ki ne naraščajo tako hitro, pravzaprav počasneje od kvadratne kazni za pospešek, je linearna funkcija. Problem premice je negativni del, ki nam bo dal neželene rezultate – pri eksponentni so vrednosti

na eni strani ničle precej majhne, tu pa še naprej padajo, zato zporabimo t.i. ReLU funkcijo, ki je za negativne vrednosti enostavno nič, za pozitivne pa narašča sorazmerno. Novo mehko vez zapišemo kot:

$$L_3' = \sum_{k=1}^{K} \left[\mathrm{ReLU}\left(\alpha_4 \left[\frac{v_0 + v_{i_k}}{2N} + \sum_{i=0}^{i_k} \frac{v_i}{N} - \sum_{j=1}^{k} l_j \right] \right) + \mathrm{ReLU}\left(\alpha_5 \left[\sum_{j=1}^{k} l_j - \sum_{i=0}^{i_k} \frac{v_i}{N} - \frac{v_{i_0} + v_{i_k}}{2N} \right] \right) \right]. \tag{11}$$



Graf 10: Primer vožnje skozi zaporedne semaforje še z ReLU vezmi. Velja opomniti, da se nekatere črte (največja α_4 in enaka oz. večja α_5) v veliki meri skrivajo za črto trde vezi, torej s tem dobimo zelo dobro rešitev. Za vse minimizacije uporabimo časovno delitev na 300 točk.

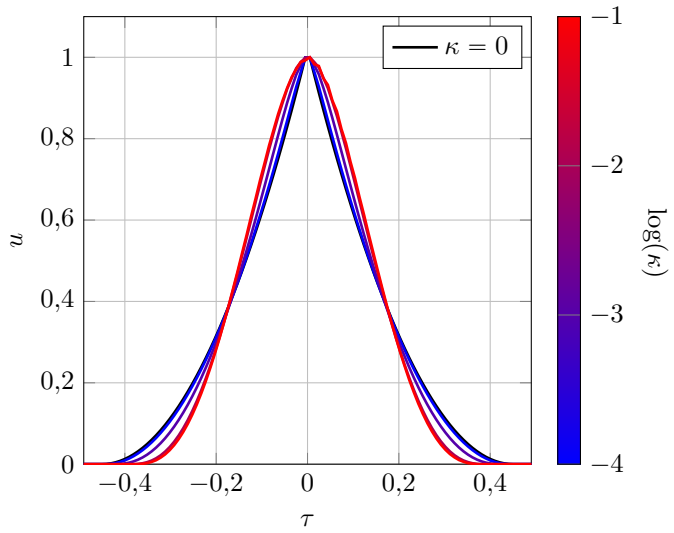
ReLU vezi nam v tem primeru res omogočajo dobro upoštevanje vezi ob minimizaciji, hkrati pa ostanejo v vlogi vezi in ne zasenčijo funkcije udobja (prevelike vrednosti pri končnih numerični natančnosti), ki jo zares želimo optimizirati. Še vedno pa lahko pri različnih nastavitvah uteži opazimo razmerje moči posameznih členov – če je $\alpha_5 > \alpha_4$, je v tem primeru pomembnejše malo hitreje prečkati semafor, kot pa priti čezenj točno ali malo kasneje, saj tako v nadaljevanju vozimo udobneje z manj pospeški. Po drugi strani za majhne α_5 lahko semaforje prečkamo malo kasneje. Pri majhnih α_4 opazimo direktno mešanje s kaznijo za nelagodje in funkcija dela le manjše popravke, ne odigra pa prave vloge vezi.

3.4 Periodične rešitve

Za periodične rešitve vemo, da so zaradi naše oblike funkcionala simetrične, zato bi lahko reševali s prostim robnim pogojem le za polovično razdaljo do semaforja, a bi tako težje zajeli pospešek ob prečkanju semaforja (ki je lahko pri nekaterih kombinacijah parametrov kar sunkovit). Da bomo enostavneje upoštevali vse pospeške in model razširili še na asimetrične funkcionale, bomo reševali cel interval in prilagodili funkcional, da bo računal tudi s pospeškom med zadnjo in prvo hitrostjo, kar zagotovi periodičnost, ne da bi morali posebej nastavljati še en robni pogoj. Da se izognemo sunku, bomo (kot pri prvi nalogi) upoštevali še sunke (spremembe pospeška) in jih kaznovali, saj so neudobni:

$$L_4 = \kappa \sum_i \left(u_{i+1} - 2u_i + u_{i-1}\right)^2 N^{-3}. \tag{12}$$

Območja indeksov vsote nismo eksplicitno navedli, saj v računalniku uporabimo 3 kopije zaporedij hitrosti, ki so medsebojno zamaknjene za 1 in tako upoštevamo tudi prehod roba (zaporedja vključujejo le točko prečkanja semaforja na levi, na desni je ne upoštevamo še enkrat).

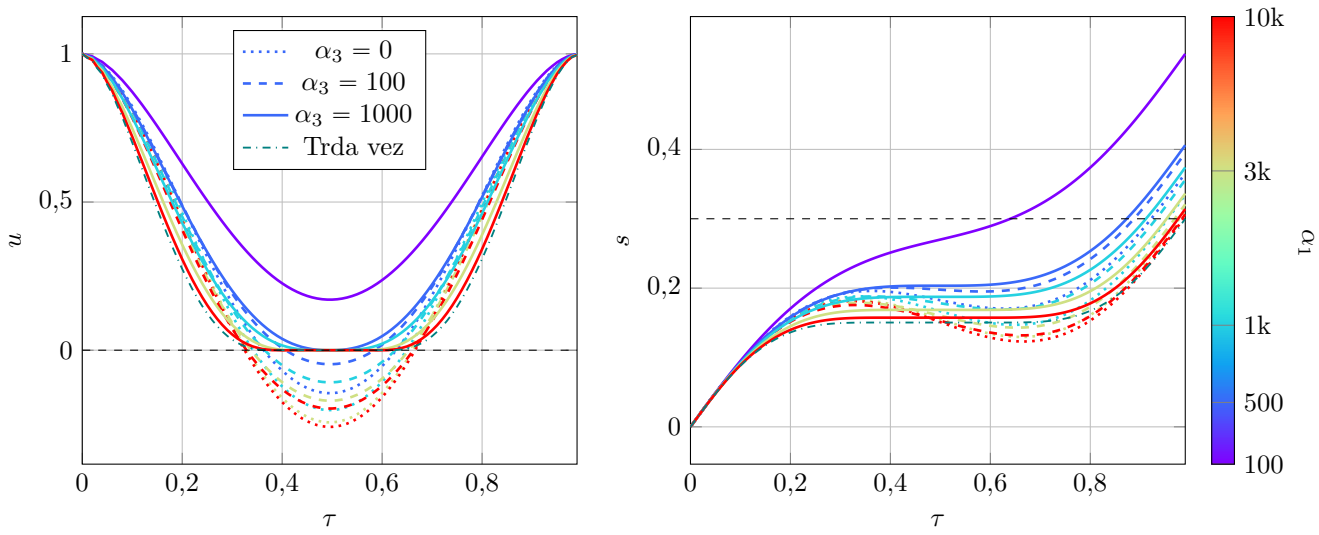


Graf 11: Periodične rešitve za različne vrednosti κ in dolžino $l = 0{,}3$. Uporabljamo trdo vez, trdo spodnjo mejo in trd robni pogoj hitrosti na semaforju.

Očitno je za vse potrebe gladke vožnje dovolj $\kappa = 0{,}05$ (pri prevelikih κ se zgodi, da zaradi diskretnega odvoda dobimo žagasto obliko v špici, zato bi morali povečati še časovno resolucijo, kar se malo že opazi na grafu pri $\kappa = 0{,}1$). Zaradi prevelikih vrednosti raje kot eksponentno funkcijo uporabimo le kvadratno kazen (L_1 brez eksponenta), in ReLU za omejitev hitrosti (podobno kot L_2, brez eksponenta in kazen le za prekoračitev, uporabljamo le $u_{\min} = 0$).

$$L_1' = \alpha_1 \left(\sum_{i=0}^{N-1} u_i N^{-1} - l \right)^2, \tag{13}$$

$$L_2' = \sum_{i=0}^{N-1} \left[\alpha_2 \cdot \mathrm{ReLU}\left(u_i - u_{\max}\right) + \alpha_3 \cdot \mathrm{ReLU}\left(u_{\min} - u_i\right) \right] \tag{14}$$



Graf 12: Vpliv uteži na minimalno hitrost in prevoženo pot pri periodičnih rešitvah z $l = 0{,}3$ in uporabljenimi kvadratnimi oz. ReLU kaznimi (kot opisano v besedilu). Razvidno je, kako velikosti uteži vplivajo na upotševanje omejitve in vezi.

Omejitev hitrosti na pozitivno nima vpliva pri prešibki uteži vezi, saj hitrost ne pade pod nič, pri večjih pa utež delno ali praktično v celoti prepreči vzvratno vožnjo, kar otežuje izpolnjevanje vezi. Šele pri zelo veliki uteži vezi sta oba pogoja dovolj dobro izpolnjena.

3.5 Omejitev pospeška in moči

Še zadnja nadgradnja je omejitev zaviranja in pospeševanja v skladu s fizikalnimi zakoni. Pri zaviranju lepenje omeji največji dovoljen pospešek, medtem ko pri pospeševanju poleg lepenja omejitev predstavlja tudi končna moč motorja. Ker sta ti dve omejitvi fizikalni in nista del želje, da se ne peljemo skozi rdečo ali vozimo vzvratno (čeprav je tudi vzvratni pospešek nemudoma po zaviranju v praksi težko izvedljiv), bomo to priložnost izkoristili za preizkus še enega tipa obteževanja kazni (lahko bi uporabili isto pri obeh, ampak potem se ničesar ne naučimo). Če naš optimizator ne bi uporabljal odvodov, bi bila dobra stopničasta funkcija, tako pa jo bomo raje aproksimirali z arctangensom, ki ima odvod in hitro zraste, vendar ne raste v neskončnost, temveč se približa dani vrednosti. Funkcij s poli ne želimo uporabljati, ker se lahko zgodi, da preskočimo pol in se znajdemo na drugi strani, kar bi privedlo do nepravilnega kaznovanja. Za strm skok uporabimo:

$$f(x) = \frac{\alpha_6}{\pi}\left(\arctan(kx) + \frac{\pi}{2}\right), \tag{15}$$

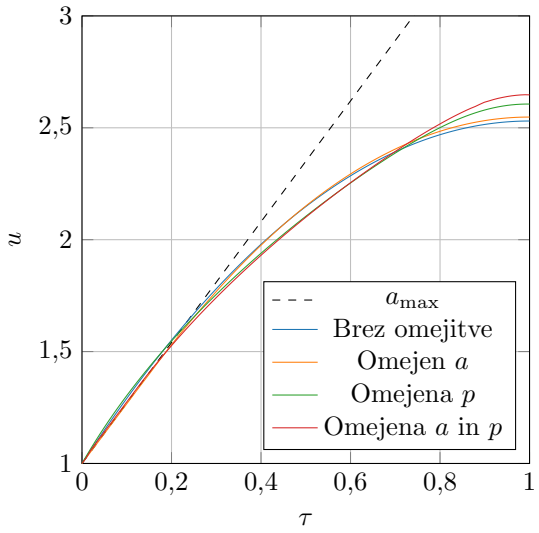
kjer je k parameter, ki je določen glede na skalo problema (kako hitro mora funkcija skočiti pri spremembi meje). Pri zaviranju omejimo diskretni pospešek, pri pospeševanju pa moč in pospešek – ne smemo preseči ne ene ne druge meje: pri nizkih hitrostih je moči dovolj (sploh če je motor električen in nima prestav), medtem ko pri višjih hitrostih pospešek omejuje moč motorja in ne več lepenje.

$$q_i = \left|\frac{u_i - u_{i-1}}{\Delta\tau} - a_{\max}\right|; \qquad w_i = u_i u_i' - p_{\max} \approx u_i\left(\frac{u_i - u_{i-1}}{\Delta\tau}\right) - p_{\max}. \tag{16}$$

Zopet nadomestimo $\Delta\tau = N^{-1}$ in uporabimo parametra q_i in w_i v funkciji kazni:

$$L_5 = \sum_{i=1}^{N}(f(q_i) + f(w_i))N^{-1}. \tag{17}$$

Utež α_6 mora biti dovolj velika, da optimizator ne bo presegel omejitve. Ostale omejitve (vez, najmanjša hitrost, robni pogoj) nastavimo v optimizatorju in spreminjamo le utež α_6 in parameter k.



Graf 13: Optimalni poteki hitrosti pri različnih kombinacijah omejitev pospeška in moči motorja. Uporabimo trdo vez $l = 2$ in trd začetni pogoj $u_0 = 1$. Za omejitev pospeška in moči uporabimo $k = 10$ in $\alpha_6 = 10$. S tem da za to razdaljo izberemo $a_{\max} = 2{,}7$ in $p_{\max} = 3{,}8$.

Izbira parametrov omejitev ni enostavna, saj minimiziramo povprečen kvadrat pospeška, zato lahko hitro pridemo do pregrobih omejitev, ki optimizator silijo v prekoračitev ene od omejitev, kar se pri tem načinu kaznovanja kaže v negladkih prehodih. Omejitev pospeška je nastavljena nekoliko pod optimalnim začetnim pospeškom, zato moramo nadoknaditi zaostanek na koncu in semafor prevozimo nekoliko hitreje. Tudi moč je omejena tako, da moramo malo bolj pospešiti na začetku, ker pri višjih hitrostih več ne moremo, se pa kljub temu nekoliko oddaljimo od optimalnega profila in končamo pri malo višji končni hitrosti (vsaj v tem primeru), ker je pospešek manjši od optimalnega v srednjem delu (ko je hitrost že višja). Ker proti koncu še ne prevozimo dovoljšne poti, moramo v zadnjem delu pospeševati bolj, kot pri optimalni rešitvi – če bi morali pospeševati tudi na zadnjem delilnem intervalu pomeni, da ob nastavljenih omejitvah ne moremo zadostiti vezi (tedaj dobimo tudi artefakte v obliki sunkov, ki jih sicer ne bi smelo biti).

4 Zaključek

V nalogi smo najprej minimizirali Thomsonov problem na krogli in primerjali učinkovitost in natančnost različnih minimizacijskih metod, nato smo poskusili še z razporejanjem nabojev po površini Möbiusovega traku, ki je geometrijsko asimetričen. V drugem delu smo ponovno reševali problem optimalne vožnje skozi semafor, le da smo tokrat uporabili diskretizacijo in poskusili omejiti različne parametre problema (tudi največjo in najmanjšo hitrost, sunke, velikost pospeška in moč motorja). Ponovno smo se ukvarjali tudi z zaporednimi semaforji in periodičnimi rešitvami in opazovali, kako obtežitve posameznih omejitev vplivajo na njihovo upoštevanje in končno rešitev.

5 Uporabljena orodja

- Python 3.13 za reševanje optimizacijskih problemov (`scipy.optimize.minimize`) in risanje grafov (skupaj s knjižnicami `scipy.spatial.convex_hull` in `trimesh` za prikaz razporeditev nabojev);
- tikzplot42 (lastna odprta knjižnica za Python) za enostavno risanje LaTeXgrafov;
- Google Gemini (Flash 3.6) za preverjanje besedila in iskanje morebitnih tipkarskih napak (popravki so bili nato narejeni ročno, zato je besedilo v celoti človeško delo);

Literatura

[1] *Thomson problem*. V: *Wikipedia*. Wikimedia Foundation. URL: `https://en.wikipedia.org/wiki/Thomson_problem`.